

CAPÍTULO 3

Usando Bison

O capítulo anterior concentrou-se apenas na flexibilidade. Neste capítulo, voltamos nossa atenção para a flexibilidade.

para bisões, embora usemos o Flex para gerar nossos analisadores lexicais. Onde o Flex reconhece

Expressões regulares, o Bison reconhece gramáticas inteiras. O Flex divide o fluxo de entrada.

em pedaços (fichas), e então o bisão pega esses pedaços e os agrupa logicamente.

Cally. Neste capítulo, vamos terminar a calculadora de mesa que começamos no Capítulo 1, começando com...

com aritmética simples e, em seguida, adicionando funções integradas, variáveis de usuário e, finalmente,

Funções definidas pelo usuário.

Como um analisador sintático do Bison compara sua entrada

O Bison recebe uma gramática que você especifica e escreve um analisador sintático que reconhece expressões válidas.

"tempos" nessa gramática. Usamos o termo frase aqui de uma forma bastante geral — para uma

Na gramática da linguagem C, as frases são programas em C sintaticamente válidos. Os programas podem

ser sintaticamente válido, mas semanticamente inválido; por exemplo, um programa em C que atribui um valor a um valor.

Converter uma string para uma variável inteira. O Bison lida apenas com a sintaxe; outras validações ficam a seu critério.

Como vimos no Capítulo 1, uma gramática é uma série de regras que o analisador sintático usa para reconhecer

entrada sintaticamente válida. Por exemplo, aqui está uma versão da gramática que usaremos mais tarde.

neste capítulo em uma calculadora:

declaração: NOME '=' expressão

expressão: NÚMERO '+' NÚMERO

| NÚMERO '-' NÚMERO

A barra vertical, |, significa que existem duas possibilidades para o mesmo símbolo; ou seja, um

A expressão pode ser uma adição ou uma subtração. O símbolo à esquerda dos dois pontos (:) é

conhecido como o lado esquerdo da régua, frequentemente abreviado como LHS, e os símbolos para o

direita é o lado direito, geralmente abreviado como RHS. Várias regras podem ter o mesmo nome.

lado esquerdo; a barra vertical é apenas uma abreviação para isso. Símbolos que realmente aparecem

Os símbolos ou tokens terminais presentes na entrada e retornados pelo analisador léxico são aqueles que são representados por símbolos terminais ou tokens.

Os elementos que aparecem no lado esquerdo de cada regra são símbolos não terminais ou não terminais.

Os símbolos terminais e não terminais devem ser diferentes; é um erro escrever uma regra com um token no lado esquerdo.

A maneira usual de representar uma frase analisada é como uma árvore. Por exemplo, se analisássemos a

Com a entrada $\text{fred} = 12 + 13$ e esta gramática, a árvore ficaria como na Figura 3-1.

Figura 3-1. Árvore de análise sintática da expressão

Neste exemplo, $12 + 13$ é uma expressão e $\text{fred} = \text{expressão}$ é uma declaração. Um bisão

O analisador sintático não cria automaticamente essa árvore como uma estrutura de dados, embora, como veremos,

Não é difícil fazer isso você mesmo. Toda gramática inclui um símbolo inicial, aquele que

Tem que estar na raiz da árvore de análise sintática. Nesta gramática, a declaração é o símbolo inicial.

As regras podem se referir a si mesmas direta ou indiretamente; essa importante capacidade torna possível...

capaz de analisar sequências de entrada arbitrariamente longas. Vamos estender nossa gramática para lidar com isso.

expressões aritméticas mais longas:

expressão: NÚMERO

| expressão + NÚMERO

| expressão – NÚMERO

Agora podemos analisar uma sequência como $\text{fred} = 14 + 23 - 11 + 7$ aplicando a expressão

regras repetidamente, como na Figura 3-2. O Bison consegue analisar regras recursivas com muita eficiência, então nós

Veremos regras recursivas em praticamente todas as gramáticas que usamos.

Análise sintática Shift/Reduce

Um analisador sintático Bison funciona procurando regras que possam corresponder aos tokens vistos até o momento.

Quando o Bison processa um analisador sintático, ele cria um conjunto de estados, cada um dos quais reflete uma possibilidade.

posição em uma ou mais regras parcialmente analisadas. À medida que o analisador lê os tokens, cada vez que

Se ler um token que não complete uma regra, ele o coloca em uma pilha interna e

48 | Capítulo 3: Utilizando Bisonte

Faça o download em Boykma.Com

O sistema muda para um novo estado que reflete o token que acabou de ler. Essa ação é chamada de mudança de estado.

Quando encontra todos os símbolos que constituem o lado direito de uma regra, ele aparece.

Os símbolos do lado direito são removidos da pilha, empurrando o símbolo do lado esquerdo para a pilha.

pilha, e alterna para um novo estado que reflete o novo símbolo na pilha. Esta ação

É chamada de redução, pois geralmente diminui o número de itens na pilha.* Quando-

Sempre que o Bison processa uma regra, ele executa o código do usuário associado a essa regra. É assim que funciona.

Você realmente faz algo com o material que o analisador sintático interpreta. Vejamos como.

O analisador sintático começa com a entrada $\text{fred} = 12 + 13$, utilizando as regras simples da Figura 3-1.

deslocando os tokens para a pilha interna um de cada vez:

Figura 3-2. Árvore de análise sintática com regras recursivas

* É possível ter regras com lados direitos vazios, caso em que uma redução resulta em mais um elemento.

item do que tinha inicialmente, mas mesmo assim chamamos isso de reduções.

Análise sintática Shift/Reduce | 49

Faça o download em Boykma.Com

Fred

$\text{fred} =$

$\text{fred} = 12$

$\text{fred} = 12 +$

$\text{fred} = 12 + 13$

Neste ponto, ele pode reduzir a expressão da regra: NÚMERO + NÚMERO, então ele remove o 12, o

além disso, e os 13 da pilha e os substitui pela expressão:

$\text{fred} = \text{expressão}$

Agora, a declaração da regra é reduzida: NOME = expressão, então ela remove fred, =, e

expressão e as substitui por declaração. Chegamos ao fim da entrada e

A pilha foi reduzida ao símbolo inicial, portanto a entrada era válida de acordo com o

gramática.

O que o analisador LALR(1) do Bison não consegue analisar

Os analisadores sintáticos Bison podem usar um dos dois métodos de análise sintática, conhecidos como LALR(1) (Look Ahead).

Da esquerda para a direita com uma previsão de um token) e GLR (Generalizado da esquerda para a direita). A maioria.

Os analisadores sintáticos usam LALR(1), que é menos poderoso, mas consideravelmente mais rápido e fácil de usar.

do que GLR. Neste capítulo, descreveremos a análise sintática LALR e deixaremos o GLR para o Capítulo 9.

Embora a análise sintática LALR seja bastante poderosa, você pode escrever gramáticas que ela não consegue processar.

dle. Não consegue lidar com gramáticas ambíguas, aquelas em que a mesma entrada pode corresponder

mais de uma árvore de análise sintática. Também não consegue lidar com gramáticas que precisam de mais de uma.

token de lookahead para indicar se correspondeu a uma regra. (Mas o bisão tem um maravilhoso

truque para lidar com os tipos mais comuns de gramáticas ambíguas, que abordaremos a seguir.

(em algumas páginas.) Considere este exemplo extremamente forçado:

frase: animal_de_carga E CARROÇA

| animal_de_trabalho E ARADO

animal_carteira: CAVALO | CABRA

animal_de_trabalho: CAVALO | BOI

Essa gramática não é ambígua, visto que existe apenas uma árvore de análise sintática possível para qualquer expressão válida.

entrada, mas o bison não consegue processá-la porque requer dois símbolos de lookahead. Em par-

Em particular, na entrada HORSE AND CART, não é possível determinar se HORSE é um animal de carroça ou um animal de tração.

animal de trabalho até ver o CART, e o bison não consegue enxergar tão longe. Se mudássemos o

primeira regra para isso:

frase: animal_carrinho CARRINHO

| arado para animais de trabalho

O bison não teria problemas, já que consegue prever um token à frente para ver se uma entrada

de CAVALO é seguido por CARROÇA, caso em que o cavalo é um animal de carroça, ou por ARADO, em

Nesse caso, trata-se de um animal de trabalho. Na prática, as regras sobre o que o bison pode realizar são

Não são tão complexos e confusos quanto podem parecer aqui. Uma das razões é que o bison sabe

exatamente quais gramáticas ele consegue analisar e quais não consegue. Se você fornecer uma que ele não consegue analisar.

Faça o download em Boykma.Com

O sistema irá te avisar, então não há problema de analisadores sintáticos excessivamente complexos falharem silenciosamente.

Outro motivo é que as gramáticas que o Bison consegue processar correspondem bastante bem a

aquelas que as pessoas realmente escrevem. Muitas vezes, trata-se de uma construção gramatical que confunde.

O termo "bison" também pode confundir as pessoas, então, se você tiver alguma liberdade no design da sua linguagem,

Você deveria considerar mudar a linguagem para torná-la mais compreensível para ambos.

bison e seus usuários. Para obter mais informações sobre análise sintática shift/reduce, consulte o Capítulo 7.

Para uma discussão sobre o que a Bison precisa fazer para transformar sua especificação em um C funcional.

Para entender melhor o programa, uma boa referência é o livro "Parsing Techniques: A Practical Guide" de Dick Grune.

Oferece o download de uma edição antiga, mas bastante adequada, em <http://www.cs.vu.nl/~dick/>

PTAPG.html.

Um analisador de bisão

Uma especificação Bison tem a mesma estrutura de três partes que uma especificação Flex. (Flex)

copiou sua estrutura do lex anterior, que copiou sua estrutura do yacc, o pred-

(A primeira seção, a seção de definições, trata das informações de controle.)

para o analisador sintático e, em geral, configura o ambiente de execução no qual o analisador sintático irá funcionar.

operar. A segunda seção contém as regras para o analisador sintático e a terceira seção é

O código C foi copiado integralmente para o programa C gerado.

Bison cria o programa em C inserindo partes em um arquivo esqueleto padrão. As regras.

são compiladas em matrizes que representam a máquina de estados que corresponde aos tokens de entrada.

As ações têm os valores \$N e @N traduzidos para C e então são colocadas em um switch.

declaração dentro de `yyparse()` que executa a ação apropriada cada vez que houver uma redução-

ção. Algumas partes do esqueleto têm múltiplas versões, das quais o bison escolhe a de-

dependendo das opções que estiverem em uso; por exemplo, se o analisador usar o recurso de locais,

Inclui código para lidar com dados de localização.

Neste capítulo, pegamos o exemplo da calculadora simples do Capítulo 1 e o ampliamos.

significativamente. Primeiro, reescrevemos o código para aproveitar alguns atalhos úteis do Bison e

alterá-lo para produzir uma estrutura de dados reutilizável em vez de calcular os valores no

voar. Mais tarde, adicionaremos uma sintaxe mais complexa para loops e funções e mostraremos como.

implementá-los em um interpretador simples.†

Árvores de Sintaxe Abstrata

Uma das estruturas de dados mais poderosas usadas em compiladores é uma árvore sintática abstrata.

(AST). No Capítulo 1, vimos uma árvore de análise sintática, uma árvore que possui um nó para cada regra usada para

Analisar a string de entrada. Na maioria das gramáticas reais, existem regras para gerenciar isso.

agrupamento, mas que não acrescentam significado ao programa. No exemplo da calculadora, as regras

† Então vamos parar por aqui; o mundo já tem muitos programas interpretadores de scripts. Para fins práticos, você

Seria muito melhor adicionar algumas extensões a uma linguagem de script já existente e bem depurada, como o Python.

Em vez de escrever mais uma, use Perl ou Lua.

Árvores de Sintaxe Abstrata | 51

Faça o download em Boykma.Com

exp: term e term: factor existem apenas para informar ao analisador a precedência relativa do

operadores. Uma AST é basicamente uma árvore de análise sintática que omite os nós irrelevantes.

regras.

Uma vez que um analisador sintático cria uma AST (Árvore Sintática Abstrata), é simples escrever rotinas recursivas que

“Caminhar” pela árvore. Veremos vários caminhantes de árvores neste exemplo.

Uma calculadora aprimorada que cria ASTs

Este exemplo é grande o suficiente para valer a pena dividi-lo em vários arquivos de origem, então vamos colocá-los em um só.

A maior parte do código C precisa estar em um arquivo separado, o que significa que também precisamos de um arquivo de cabeçalho C.

Declare as rotinas e estruturas de dados utilizadas nos diversos arquivos (Exemplo 3-1).

Exemplo 3-1. Calculadora que constrói uma AST: cabeçalho fb3-1.h

```
/*
```

```
* Declarações para uma calculadora fb3-1
```

```
*/
```

```
/* interface para o analisador léxico */
```

```
extern int yylineno; /* do analisador léxico */
```

```
void yyerror(char *s, ...);
```

```
/* nós na árvore de sintaxe abstrata */
```

```
struct ast {
```

```
    tipo de nó inteiro;
```

```
    struct ast *l;
```

```
    struct ast *r;
```

```
};
```

```
struct numval {
```

```
    int nodetype; /* tipo K para constante */
```

número duplo;

};

/* construir uma AST */

struct ast *newast(int nodetype, struct ast *l, struct ast *r);

struct ast *newnum(double d);

/* avaliar uma AST */

double eval(struct ast *);

/* excluir e liberar uma AST */

void treefree(struct ast *);

A variável `yylineno` e a rotina `yyerror` são familiares do exemplo `flex`. Nosso

A função `yyerror` foi ligeiramente aprimorada para aceitar múltiplos argumentos, no estilo da função `printf`.

A AST consiste em nós, cada um com um tipo de nó. Nós diferentes têm tipos de nó diferentes.

campos, mas por enquanto temos apenas dois tipos, um que possui ponteiros para até dois subnós

e uma que contém um número. Duas rotinas, `newast` e `newnum`, criam nós AST;

52 | Capítulo 3: Utilizando Bison

Faça o download em Boykma.Com

``eval`` percorre uma AST e retorna o valor da expressão que ela representa; e ``treefree``

Percorre uma AST e exclui todos os seus nós.

Exemplo 3-2. Analisador sintático Bison para calculadora AST

```
/* calculadora com AST */
```

```
%{
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
# incluir "fb3-1.h"
```

```
%}
```

```
%união {
```

```
struct ast *a;
```

```
duplo d;
```

```
}
```

```
/* declarar tokens */
```

```
%token <d> NÚMERO
```

```
%token EOL
```

```
%tipo <a> termo fator exponencial
```

O Exemplo 3-2 mostra o analisador sintático Bison para a calculadora AST. A primeira seção do

O analisador sintático usa a construção %union para declarar os tipos a serem usados nos valores dos símbolos em

o analisador sintático. Em um analisador sintático Bison, cada símbolo, tanto tokens quanto não terminais, pode ter um

valor associado a ele. Por padrão, os valores são todos inteiros, mas programas úteis

geralmente precisam de valores mais sofisticados. A construção %union, como o próprio nome sugere,

é usado para criar uma declaração de união em linguagem C para valores de símbolos. Neste caso, o

A união possui dois membros: a, que é um ponteiro para uma AST, e d, que é um double.

número de precisão.

Uma vez definida a união, precisamos dizer ao bisão quais símbolos têm quais tipos de valores.

colocando o nome apropriado da união entre colchetes angulares (< >). O token

NUMBER, que representa os números na entrada, tem o valor <d> para armazenar o valor de

o número. A nova declaração %type atribui o valor <a> a exp, factor e term.

que usaremos à medida que construimos nossa AST.

Você não precisa declarar um tipo para um token nem declarar um não-terminal, se não quiser.

Use o valor do símbolo. Se houver uma %union nas declarações, o Bison fornecerá um

Ocorrerá um erro se você tentar usar o valor de um símbolo que não possui um tipo atribuído.

Lembre-se de que qualquer regra sem código de ação explícito recebe a ação padrão \$\$ =

US\$ 1; e o bisão reclamará se o símbolo do lado esquerdo tiver um tipo e o símbolo do lado direito não tiver.

têm o mesmo tipo.

%%

lista de cálculos: /* nada */

| calclist exp EOL {

printf("= %4.4g\n", eval(\$2)); avalia e imprime a AST

treefree(\$2); libera a AST

Uma calculadora aprimorada que cria ASTs | 53

Faça o download em Boykma.Com

printf("> ");

}

| calclist EOL { printf("> "); } /* linha em branco ou um comentário */

;

exp: fator

| exp '+' factor { \$\$ = newast('+', \$1,\$3); }

| exp '-' factor { \$\$ = newast('-', \$1,\$3);}

;

fator: termo

| factor '*' term { \$\$ = newast('*', \$1,\$3); }

| factor '/' term { \$\$ = newast('/', \$1,\$3); }

;

termo: NÚMERO { \$\$ = novonum(\$1); }

| '|' termo { \$\$ = newast('|', \$2, NULL); }

```
| '(' exp ')' { $$ = $2; }
```

```
| '-' termo { $$ = newast('M', $2, NULL); }
```

```
;
```

```
%%
```

Fichas de Caracteres Literais

A seção de regras apresenta duas mudanças significativas em relação à versão do Capítulo 1. Uma delas é que

As regras agora usam tokens literais para os operadores. Em vez de atribuir um valor literal a cada token,

Além do nome, também é possível usar um caractere entre aspas simples como um token, com o valor ASCII.

do token ser o número do token. (Bison inicia a numeração de tokens nomeados em

258, então não há problema de colisões.) Por convenção, os tokens de caracteres literais são

usado para representar tokens de entrada compostos pelo mesmo caractere; por exemplo, o token

O caractere '+' representa o token de entrada +, portanto, na prática, ele é usado apenas para pontuação e

operadores. Não há necessidade de declarar tokens de caracteres literais, a menos que você precise declarar

o tipo de seus valores.

A outra alteração no analisador sintático é que ele cria uma AST para cada expressão em vez de...

avaliando-o em tempo real. Nesta versão da calculadora, criamos uma AST para cada

Expressão e, em seguida, avaliamos a AST, imprimimos o resultado e liberamos a AST. Chamamos

newast() para criar cada nó na AST. Cada nó possui um tipo de operador, que é

geralmente é o mesmo que o nome do token. Observe que o sinal de menos unário cria um nó do tipo

M para distingui-la da subtração binária.

Exemplo 3-3. Analisador léxico para calculadora AST

```
/* reconhecer tokens para a calculadora */
```

```
%option noyywrap nodefault yylineno
```

```
%{
```

```
# incluir "fb3-1.h"
```

```
#include "fb3-1.tab.h"
```

```
%}
```

54 | Capítulo 3: Utilizando Bison

Faça o download em Boykma.Com

```
/* expoente de ponto flutuante */
```

```
EXP ([Ee][-+]?[0-9]+)
```

```
%%
```

```
"+" |
```

```
"-" |
```

```
"*" |
```

```
"/" |
```

```

"|" |
 "(" |
 ")" { return yytext[0]; }
 [0-9]+ "." [0-9]* {EXP}? |
 "."? [0-9]+ {EXP}? { yylval.d = atof(yytext); return NUMBER; }
 \n { return EOL; }
 "//" .*
 [ \t] { /* ignorar espaços em branco */ }
 . { yyerror("Caractere misterioso %c\n", *yytext); }
 %%

```

O analisador léxico, mostrado no Exemplo 3-3, é um pouco mais simples do que a versão do Capítulo 1. Nós

Use uma expressão idiomática comum para os operadores de um único caractere, tratando-os todos da mesma maneira.

regra que retorna yytext[0], o próprio caractere, como token. Ainda temos nomes.

para NUMBER e EOL. Também lidamos com números de ponto flutuante, usando uma versão do padrão.

partindo do Capítulo 2, altere a representação interna dos números para double.

Como yylval agora é uma união, o valor double precisa ser atribuído a yylval.d. (Flex faz isso)

(Não automatiza o gerenciamento de valores de tokens como o Bison faz.)

Exemplo 3-4. Rotinas em C para calculadora AST

```

#include <stdio.h>

#include <stdlib.h>

#include <stdarg.h>

# incluir "fb3-1.h"

struct ast *

newast(int nodetype, struct ast *l, struct ast *r)

{

struct ast *a = malloc(sizeof(struct ast));

se(!a) {

yyerror("sem espaço");

sair(0);

}

a->nodetype = nodetype;

a->l = l;

a->r = r;

retornar a;

}

struct ast *

novonum(duplo)

{

struct numval *a = malloc(sizeof(struct numval));

```

Faça o download em Boykma.Com

```
se(!a) {  
yyerror("sem espaço");  
sair(0);  
}  
a->nodetype = 'K';  
a->número = d;  
retornar (struct ast *)a;  
}
```

Finalmente, há um arquivo de rotinas chamadas pelo analisador sintático, mostrado no Exemplo 3-4.‡ Primeiro

Existem duas rotinas para criar nós da AST: alocando um nó com malloc e preenchendo-o.

Os nós da AST têm um tipo de nó como primeiro campo, portanto, uma análise da árvore pode determinar o tipo de nó.

É atravessar caminhando.

dobro

```
eval(struct ast *a)  
{  
valor duplo v; valor calculado desta subárvore  
switch(a->nodetype) {  
caso 'K': v = ((struct numval *)a)->número; interromper;  
caso '+': v = eval(a->l) + eval(a->r); break;
```

```

    caso '-': v = eval(a->l) - eval(a->r); break;
    caso '*': v = eval(a->l) * eval(a->r); break;
    caso '/': v = eval(a->l) / eval(a->r); break;
    caso '|': v = eval(a->l); se(v < 0) v = -v; interromper;
    caso 'M': v = -eval(a->l); interromper;
    padrão: printf("erro interno: nó inválido %c\n", a->nodetype);
}

retornar v;
}

vazio

treefree(struct ast *a)
{
    switch(a->nodetype) {
        /* duas subárvores */
        caso '+':
        caso '-':
        caso '*':
        caso '/':
            treefree(a->r);

        /* uma subárvore */
        caso '|':
        caso 'M':
    }
}

```

```
treefree(a->l);
```

‡ Estes poderiam ter ido para a terceira seção do analisador sintático, mas é mais fácil depurar seu programa se você não os incluir.

Inclua muito código extra no arquivo do analisador sintático.

56 | Capítulo 3: Utilizando Bison

Faça o download em Boykma.Com

```
/* nenhuma subárvore */
```

```
caso 'K':
```

```
livre(a);
```

```
quebrar;
```

```
padrão: printf("erro interno: liberar nó ruim %c\n", a->nodetype);
```

```
}
```

```
}
```

Em seguida, temos as duas rotinas de caminhada na árvore. Cada uma delas realiza o que é conhecido como uma profundidade-

primeiro percurso da árvore, visitando recursivamente as subárvores de cada nó e depois o

o próprio nó. A rotina eval retorna o valor da árvore ou subárvore de cada chamada, e

A Treefree não precisa devolver nada.

```
vazio
```

```
yyerror(char *s, ...)
```

```
{
```

```
listar ap;
```

```

va_start(ap, s);
fprintf(stderr, "%d: erro: ", yylineno);
vfprintf(stderr, s, ap);
fprintf(stderr, "\n");
}

inteiro
principal()
{
printf("> ");
retornar yyparse();
}

```

Finalmente, temos `yyerror` e `main`. Esta versão de `yyerror` usa argumentos variáveis (varargs) para aceitar um argumento.

Lista de argumentos no estilo printf, que se mostra conveniente ao gerar erros.

mensagens.

Construindo a Calculadora AST

Este programa agora possui três arquivos de código-fonte e um arquivo de cabeçalho, então, é claro, usamos o comando `make` para...

construa-o.

```
fb3-1: fb3-1.l fb3-1.y fb3-1.h
```

```
bisão -d fb3-1.y
```

```
flex -ofb3-1.lex.c fb3-1.l
```

cc -o \$@ fb3-1.tab.c fb3-1.lex.c fb3-1funcs.c

Observe a flag -o do flex. O Bison nomeia automaticamente o arquivo C gerado para corresponder.

o arquivo .y, mas o Flex sempre chama seu arquivo C de lex.yy.c, a menos que você especifique o contrário.

Uma calculadora aprimorada que cria ASTs | 57

Faça o download em Boykma.Com

Conflitos de turno/redução e precedência do operador

O analisador de expressões usa três símbolos diferentes, exp, factor e term, para definir o

precedência e associatividade de operadores. Embora este analisador ainda seja razoavelmente legítimo-

À medida que as gramáticas adicionam mais operadores com mais níveis de precedência, elas se tornam

cada vez mais difícil de ler e manter. Bison oferece uma maneira inteligente de descrever o

precedência separadamente das regras da gramática, o que torna a gramática e

O analisador sintático ficará menor e mais fácil de manter. Primeiro, vamos fazer com que todas as expressões usem exp.

símbolos:

%type <a> exp

%%

...

exp: exp '+' exp { \$\$ = newast('+', \$1,\$3); }

```

| exp '-' exp { $$ = newast('-', $1,$3);}
| exp '*' exp { $$ = newast('*', $1,$3); }
| exp '/' exp { $$ = newast('/', $1,$3); }
| '|' exp { $$ = newast('|', $2, NULL); }
| '(' exp ')' { $$ = $2; }
| '-' exp { $$ = newast('M', $2, NULL); }
| NÚMERO { $$ = novonum($1); }
;

%%

```

Mas essa gramática tem um problema: ela é extremamente ambígua. Por exemplo, a entrada

$2+3*4$ pode significar $(2+3)*4$ ou $2+(3*4)$, e a entrada $3-4-5-6$ pode significar $3-(4-(5-6))$

ou $(3-4)-(5-6)$ ou qualquer uma de muitas outras possibilidades. A Figura 3-3 mostra as duas possibilidades.

analisa para $2+3*4$.

Figura 3-3. Duas árvores de análise sintática de expressões

58 | Capítulo 3: Utilizando Bison

Faça o download em Boykma.Com

Se você compilar essa gramática como está, o Bison informará que existem 24 operações de shift/reduce.

conflitos, que são estados em que não é possível determinar se o token deve ser movido para o

Empilhe ou reduza uma regra primeiro. Por exemplo, ao analisar $2+3*4$, o analisador percorre o seguinte caminho:

estes passos (aqui abreviamos exp como E):

2 turno NÚMERO

E reduz E

→ NÚMERO

E + shift +

E + 3 mudança NÚMERO

E + E reduz E

→ NÚMERO

Neste ponto, o analisador examina o $*$ e pode simplificar $2+3$ usando:

exp: exp '+' exp

para uma expressão ou deslocamento do $*$, esperando poder reduzir:

exp: exp '*' exp

mais tarde.

O problema é que não contamos aos bisontes sobre a precedência e a associatividade de

Os operadores. A precedência controla quais operadores são executados primeiro em uma expressão.

Tradição matemática e de programação (que remonta a antes do primeiro compilador Fortran)

(em 1956) afirma que a multiplicação e a divisão têm precedência sobre a adição e a subtração.

tração, então $a+b*c$ significa $a+(b*c)$, e $d/e-f$ significa $(d/e)-f$. Em qualquer gramática de expressões,

Os operadores são agrupados em níveis de precedência, do mais baixo ao mais alto. O número total-

A quantidade de níveis depende da linguagem. A linguagem C é notória por ter muitos níveis.

níveis de precedência, num total de 15 níveis.

A associatividade controla o agrupamento de operadores no mesmo nível de precedência. Operadores

podem se agrupar à esquerda, por exemplo, $a-b-c$ em C significa $(a-b)-c$, ou à direita, por exemplo-

Por exemplo, $a=b=c$ em C significa $a=(b=c)$. Em alguns casos, os operadores não se agrupam; por exemplo,

Por exemplo, em Fortran, $A.LE.B.LE.C$ é inválido.

Existem duas maneiras de especificar precedência e associatividade em uma gramática, implicitamente.

e explicitamente. Até agora, especificamos isso implicitamente, usando não terminais separados.

símbolos para cada nível de precedência. Esta é uma maneira perfeitamente razoável de escrever uma gramática.

mar, e se o Bison não tivesse regras de precedência explícitas, essa seria a única maneira.

Mas o Bison também permite especificar a precedência explicitamente. Podemos adicionar essas linhas ao

Seção de definição para indicar como resolver os conflitos:

```
%left '+' '-'
```

%esquerda '*' '/'

%nonassoc '|' LUZ

%type <a> exp

%%

...

exp: exp '+' exp { \$\$ = newast('+', \$1,\$3); }

| exp '-' exp { \$\$ = newast('-', \$1,\$3); }

Conflitos de turno/redução e precedência do operador | 59

Faça o download em Boykma.Com

| exp '*' exp { \$\$ = newast('*', \$1,\$3); }

| exp '/' exp { \$\$ = newast('/', \$1,\$3); }

| '|' exp { \$\$ = newast('|', \$2, NULL); }

| '(' exp ')' { \$\$ = \$2; }

| '-' exp %prec UMINUS { \$\$ = newast('M', NULL, \$2); }

| NÚMERO { \$\$ = novonum(\$1); }

;

Cada uma dessas declarações define um nível de precedência, com a ordem do %left,

As declarações %right e %nonassoc definem a ordem de precedência, da mais baixa para a mais alta.

mais alta. Eles dizem aos bisontes que + e - são associativos à esquerda e têm a menor precedência.

nível; * e / são associativos à esquerda e têm um nível de precedência mais alto; e | e UMINUS, um

pseudotokens que representam o sinal menos unário, não possuem associatividade e estão no nível mais alto.

precedência. (Não temos nenhum operador associativo à direita aqui, mas se tivéssemos, eles seriam

(use %right.) O Bison atribui a cada regra a precedência do token mais à direita na lista à direita.

lado esquerdo; se esse token não tiver precedência atribuída, a regra não terá precedência sobre ele.

próprio. Quando o bisão se depara com um conflito de deslocamento/redução, ele consulta a tabela de precedência.

E se todas as regras envolvidas no conflito tiverem uma precedência atribuída, ela será usada.

para resolver o conflito.

Em nossa gramática, todos os conflitos ocorrem nas regras da forma $exp \ OP \ exp$, então configurando

A precedência dos quatro operadores permite que todos os conflitos sejam resolvidos. Este analisador sintático

O uso de precedência resulta em um sistema ligeiramente menor e mais rápido do que aquele com regras adicionais para

precedência implícita, já que possui menos regras para reduzir.

A regra de negação inclui %prec UMINUS. O único operador nesta regra é -, que

tem baixa precedência, mas queremos que o operador menos unário tenha precedência maior que o operador menos múltiplo.

plicação em vez de inferior. O %prec indica ao Bison para usar a precedência de UMINUS para

esta regra.

Quando não usar regras de precedência

Você pode usar regras de precedência para corrigir qualquer conflito de deslocamento/redução que ocorra na gramática.

Essa geralmente é uma péssima ideia. Em gramáticas de expressões, a causa dos conflitos é fácil.

Para entender, e o efeito das regras de precedência é claro. Em outras situações, precedência-

As regras de correção resolvem problemas de shift/reduce, mas podem ser extremamente difíceis de entender.

exatamente qual o efeito que elas têm na gramática.

Use precedência apenas em duas situações: em gramáticas de expressões e para resolver o

Conflito de “else pendente” em gramáticas para construções linguísticas if/then/else. (Veja a seção-

(Consulte a seção “IF/THEN/ELSE” na página 185 para exemplos do último.) Caso contrário, se possível,

Você deve corrigir a gramática para eliminar o conflito. Lembre-se de que conflitos significam que

O Bison não consegue criar um analisador sintático para uma gramática, provavelmente porque ela é ambígua. Isso significa

Existem várias interpretações possíveis para a mesma entrada, e o analisador sintático criado por Bison é um exemplo disso.

Escolha uma delas. Exceto nos dois casos anteriores, isso geralmente indica um problema.

na sua definição de idioma. Em alguns casos, se uma gramática for ambígua para o bisão, é

quase certamente ambíguo também para os humanos. Consulte o Capítulo 8 para obter mais informações sobre

60 | Capítulo 3: Utilizando Bisonte

Faça o download em Boykma.Com

Encontrar e reparar conflitos, bem como os recursos avançados do Bison que permitem usar

Gramáticas ambíguas, se você realmente quiser.

Uma calculadora avançada

O exemplo final deste capítulo amplia a calculadora para torná-la pequena, mas um tanto...

Compilador realista. Adicionaremos variáveis nomeadas e atribuições; expressões de comparação-

ções (maior, menor, igual, etc.); controle de fluxo com if/then/else e while/do; integrado

e funções definidas pelo usuário; e um pouco de recuperação de erros. A versão anterior do

A calculadora não aproveitou muito a representação AST de expressões, mas

Neste caso, a AST (Árvore Sintática Abstrata) é fundamental para a implementação do controle de fluxo e das funções do usuário.

Aqui está um exemplo de como definir uma função de usuário e, em seguida, chamá-la usando uma função integrada.

função como um dos argumentos:

> seja $\text{avg}(a,b) = (a+b)/2$;

Média definida

```
> média(3, raiz(25))
```

```
= 4
```

Como antes, começamos com as declarações, mostradas no Exemplo 3-5.

Exemplo 3-5. Cabeçalho da calculadora avançada fb3-2.h

```
/*  
  
* Declarações para uma calculadora fb3-1  
  
*/  
  
/* interface para o analisador léxico */  
extern int yylineno; /* do analisador léxico */  
void yyerror(char *s, ...);  
  
/* tabela de símbolos */  
struct symbol { /* um nome de variável */  
    char *nome;  
    valor duplo;  
    struct ast *func; /* declaração para a função */  
    struct symlist *syms; /* lista de argumentos fictícios */  
};  
  
/* tabela simbólica simples de tamanho fixo */  
#definir NHASH 9997  
struct symbol symtab[NHASH];  
struct symbol *lookup(char*);
```

```
/* lista de símbolos, para uma lista de argumentos */
```

```
struct symlist {  
    struct symbol *sym;  
    struct symlist *next;  
};
```

Uma calculadora avançada | 61

Faça o download em Boykma.Com

```
struct symlist *newsymlist(struct symbol *sym, struct symlist *next);  
void symlistfree(struct symlist *sl);
```

A tabela de símbolos foi adaptada daquela utilizada no capítulo anterior. Na calculadora,

Cada símbolo pode potencialmente ser tanto uma variável quanto uma função definida pelo usuário.

O campo `value` armazena o valor do símbolo como uma variável, enquanto o campo `func` aponta para a AST (Árvore Sintática Abstrata) para...

o código do usuário para a função, e syms aponta para uma lista encadeada de argumentos fictícios,

que são símbolos em si mesmos. (No exemplo anterior, avg é a função, e a e

b são os argumentos fictícios.) As funções C newsymlist e symlistfree criam e

Libertem-nos.

```
/* tipos de nó
```

```
* + - * / |
```

* 0-7 operações de comparação, codificadas em bits: 04 igual, 02 menor, 01 maior

* Menos lunar

* Lista de expressões ou declarações em L

* Declaração SE

* Declaração W WHILE

* Referência do símbolo N

* = atribuição

* Lista de símbolos

* Chamada de função integrada F

* Chamada de função do usuário C

*/

enum bifs { /* funções internas */

B_sqrt = 1,

B_exp,

B_log,

Impressão B

};

/* nós na árvore de sintaxe abstrata */

/* todos têm um tipo de nó inicial comum */

struct ast {

tipo de nó inteiro;

struct ast *l;

```

struct ast *r;

};

struct fncall { /* função embutida */
int nodetype; /* tipo F */
struct ast *l;
enum bifs functype;
};

struct ufncall { /* função do usuário */
int nodetype; /* tipo C */
struct ast *l; /* lista de argumentos */
struct symbol *s;
};

struct fluxo {

62 | Capítulo 3: Utilizando Bison
Faça o download em Boykma.Com

int nodetype; /* tipo I ou W */
struct ast *cond; /* condição */
struct ast *tl; /* então ramifica ou executa a lista */
struct ast *el; /* ramo else opcional */
};

struct numval {
int nodetype; /* tipo K */

```

número duplo;

};

struct symref {

int nodetype; /* tipo N */

struct symbol *s;

};

struct symasgn {

int nodetype; /* tipo = */

struct symbol *s;

struct ast *v; /* valor */

};

/* construir uma AST */

struct ast *newast(int nodetype, struct ast *l, struct ast *r);

struct ast *newcmp(int cmptype, struct ast *l, struct ast *r);

struct ast *newfunc(int functype, struct ast *l);

struct ast *newcall(struct symbol *s, struct ast *l);

struct ast *newref(struct symbol *s);

struct ast *newasgn(struct symbol *s, struct ast *v);

struct ast *newnum(double d);

struct ast *newflow(int nodetype, struct ast *cond, struct ast *tl, struct ast *tr);

/* define uma função */

```
void dodef(struct symbol *name, struct symlist *syms, struct ast
*stmts);

/* avaliar uma AST */

double eval(struct ast *);

/* excluir e liberar uma AST */

void treefree(struct ast *);

/* interface para o analisador léxico */

extern int yylineno; /* do analisador léxico */

void yyerror(char *s, ...);
```

Esta versão possui consideravelmente mais tipos de nós na AST. Como antes, cada tipo de

O nó começa com um tipo de nó que o código de percurso na árvore pode usar para determinar o tipo de nó.

É isso mesmo. O nó AST básico também é usado para comparações, com cada tipo de comparação.

(menor, menos-igual, igual, etc.) sendo um tipo diferente, e para listas de expressões.

§ Este é um truque clássico de C que não funciona em C++. Se você escrever seu analisador sintático em C++, como descrevemos em

No Capítulo 9, você precisará usar uma união explícita de estruturas dentro da estrutura AST para obter um resultado semelhante.

Uma calculadora avançada | 63

Faça o download em Boykma.Com

As funções internas possuem um nó fncall com a AST do argumento (cada uma das funções internas possui um nó fncall).

recebe um argumento) e um enum que indica qual função interna é. Existem três

Funções padrão, como raiz quadrada, exponencial e logaritmo, além de imprimir, uma função que imprime seus argumentos.

e retorna o argumento como seu valor. Chamadas para funções do usuário possuem um nó `ufncall`.

com um ponteiro para a função, que é uma entrada na tabela de símbolos, e uma AST, que

é uma lista dos argumentos.

Expressões de controle de fluxo `if/then/else` e `while/do` utilizam um nó de fluxo com o controle.

expressão, o ramo "então" ou lista "faça" e o ramo "senão" opcional.

As constantes são ``numval`` como antes; as referências a símbolos são ``symref`` com um ponteiro para o símbolo.

símbolo na tabela de símbolos; e atribuições são `symasn` com um ponteiro para o símbolo

a ser atribuído e a AST do valor a ser atribuído a ele.

Toda AST possui um valor. O valor de um `if/then/else` é o valor do ramo escolhido; o

O valor de `while/do` é o último valor da lista do `do`; e o valor de uma lista de expressões é

a última expressão.|| Finalmente, temos procedimentos em C para criar cada tipo de nó AST.

e um procedimento para criar uma função definida pelo usuário.

Analizador de calculadora avançada

O exemplo 3-6 mostra o analisador sintático para a calculadora AST avançada.

Exemplo 3-6. Analisador de calculadora avançada fb3-2.y

```
/* calculadora com AST */

%{

#include <stdio.h>

#include <stdlib.h>

# incluir "fb3-2.h"

%}

%união {

struct ast *a;

duplo d;

struct symbol *s; /* qual símbolo */

struct symlist *sl;

int fn; /* qual função */

}

/* declarar tokens */

%token <d> NÚMERO

%token <s> NOME

%token <fn> FUNC

%token EOL
```

Este projeto é vagamente baseado na antiga linguagem de programação de sistemas BLISS. Seria perfeitamente adequado.

É possível projetar uma AST que trate expressões e instruções separadamente, mas a implementação é um

Assim fica um pouco mais simples.

64 | Capítulo 3: Utilizando o Bison

Faça o download em Boykma.Com

```
%token SE ENTÃO SENÃO ENQUANTO FAÇA DEIXE
```

```
%nonassoc <fn> CMP
```

```
%right '='
```

```
%left '+' '-'
```

```
%esquerda '*' '/'
```

```
%nonassoc '|' LUZ
```

```
%type <a> exp stmt list explist
```

```
%type <sl> lista simbólica
```

```
%início da lista de cálculos
```

```
%%
```

A %union aqui define muitos tipos de valores simbólicos, o que é típico em bisões realistas.

analísadores sintáticos. Além de um ponteiro para uma AST e um valor numérico, um valor pode ser um ponteiro.

para a tabela de símbolos de um símbolo de usuário, uma lista de símbolos ou um subtipo de uma comparação.

ou token de função. (Usamos a palavra símbolo aqui de forma um tanto confusa, tanto para

nomes usados na gramática bison e nomes que o usuário digita no compilado

programa. Usaremos o símbolo do usuário para este último quando o contexto não for claro.)

Existe um novo token FUNC para as funções integradas, cujo valor indica qual delas.

função e seis palavras reservadas, IF a LET. O token CMP é qualquer uma das seis palavras reservadas.

operadores de parison, com o valor indicando qual operador. (Este truque de usar um

O token para vários operadores sintaticamente semelhantes ajuda a manter o tamanho do

gramática.)

A lista de declarações de precedência começa com os novos operadores CMP e =.

Uma declaração `%start` identifica a regra de nível superior, portanto não precisamos colocá-la no início.

início do analisador sintático.

Sintaxe da instrução da calculadora

declaração: SE exp ENTÃO lista { \$\$ = novofluxo('I', \$2, \$4, NULL); }

| SE exp ENTÃO lista SENÃO lista { \$\$ = newflow('I', \$2, \$4, \$6); }

| ENQUANTO exp FAÇA lista { \$\$ = newflow('W', \$2, \$4, NULL); }

| exp

;

lista: /* nada */ { \$\$ = NULL; }

```
| stmt ';' lista { se ($3 == NULL)
```

```
$$ = $1;
```

```
outro
```

```
$$ = newast('L', $1, $3);
```

```
}
```

```
;
```

Nossa gramática distingue entre enunciados (stmt) e expressões (exp). Um enunciado-

O mento é um fluxo de controle (if/then/else ou while/do) ou uma expressão. O if e

enquanto as declarações recebem listas de declarações, com cada declaração na lista sendo seguida

Uma calculadora avançada | 65

Faça o download em Boykma.Com

por um ponto e vírgula. Cada regra que corresponde a uma declaração chama uma rotina para construir uma aproximação.

Nó AST privado.

O design da sintaxe aqui é em grande parte arbitrário, e essa é uma das vantagens de usar...

A grande vantagem do Bison em criar um analisador sintático é a facilidade com que se pode experimentar variações. A interação

Entre os detalhes da sintaxe, as nuances podem ser bastante sutis; se, por exemplo, a definição de lista tivesse

Se você colocasse ponto e vírgula entre as frases em vez de depois delas, a gramática ficaria incorreta.

bíquo, a menos que a gramática também adicionasse os tokens de fechamento FI e ENDDO para indicar o fim.

de instruções if/then e while/do.

A definição de lista é recursiva à direita, isto é, stmt ; lista em vez de lista stmt ;.

Não faz diferença alguma em relação ao idioma reconhecido, mas facilita a construção.

a lista de declarações encadeadas da cabeça para a cauda, em vez da cauda para a cabeça. Cada vez

a regra de lista stmt; é reduzida, ela cria um link que adiciona a declaração ao cabeçalho de

a lista até agora. Se a regra fosse list stmt ;, a declaração precisaria ir no final.

da lista, o que exigiria uma lista circularmente encadeada mais complexa ou então

invertendo a lista no final (como fizemos com a lista de referências no Capítulo 1).

Uma desvantagem da recursão à direita em relação à recursão à esquerda é que a recursão à direita coloca tudo em prática.

das instruções ainda não reduzidas na pilha do analisador sintático e, em seguida, reduz todas elas em

o final da lista, enquanto a recursão à esquerda constrói a lista uma instrução por vez como entrada.

é analisado. Em uma situação como essa, onde é improvável que a lista tenha mais do que alguns itens.

Não importa o tamanho da lista, mas em um idioma onde a lista pode ter milhares de itens...

itens, vale a pena criar a lista com uma regra recursiva à esquerda e depois invertê-la para evitar

estouro de pilha do analisador sintático. Alguns programadores também acham a recursão à esquerda mais fácil de depurar.

pois tende a produzir resultados após cada instrução, em vez de todos de uma vez no final.

Sintaxe de expressão da calculadora

```
exp: exp CMP exp { $$ = newcmp($2, $1, $3); }  
| exp '+' exp { $$ = newast('+', $1,$3); }  
| exp '-' exp { $$ = newast('-', $1,$3);}   
| exp '*' exp { $$ = newast('*', $1,$3); }  
| exp '/' exp { $$ = newast('/', $1,$3); }  
| '|' exp { $$ = newast('|', $2, NULL); }  
| '(' exp ')' { $$ = $2; }  
| '-' exp %prec UMINUS { $$ = newast('M', $2, NULL); }  
| NÚMERO { $$ = novonum($1); }  
| NOME { $$ = newref($1); }  
| NOME '=' exp { $$ = newasgn($1, $3); }  
| FUNC '('explista')' { $$ = novafunc($1, $3); }  
| NOME '(' explist ')' { $$ = newcall($1, $3); }  
;
```

lista de explicações: exp

```
| exp ',' explist { $$ = newast('L', $1, $3); }
```

;

lista_simbólica: NOME { \$\$ = nova_lista_simbólica(\$1, NULL); }

| NOME ',' lista_simbólica { \$\$ = nova_lista_simbólica(\$1, \$3); }

;

66 | Capítulo 3: Utilizando Bison

Faça o download em Boykma.Com

A sintaxe de expressão é uma versão ligeiramente expandida da sintaxe de expressão em

exemplo anterior. Uma nova regra para CMP lida com os seis operadores de comparação, usando o

valor do CMP para determinar qual operador foi usado, e uma regra para atribuições cria um

nó de atribuição.

Existem regras separadas para funções internas identificadas por um nome reservado (FUNC) e

Funções do usuário identificadas por um símbolo de usuário (NOME).

Uma regra para `explist`, uma lista de expressões, constrói uma AST (Árvore Sintática Abstrata) das expressões usadas para a

argumentos reais para uma chamada de função. Uma regra separada para symlist, uma lista de símbolos, é criada.

Uma lista encadeada de símbolos para os argumentos fictícios em uma definição de função. Ambos são

A recursão à direita facilita a construção da lista na ordem desejada.

Gramática da calculadora de alto nível

lista de cálculos: /* nada */

```

| calclist stmt EOL {
printf("= %4.4g\n> ", eval($2));
treefree($2);
}

| calclist LET NAME '(' symlist ')' '=' list EOL {
urso($3, $5, $8);
printf("Definido %s\n> ", $3->name); }

| erro de lista de cálculo EOL { yyerrok; printf("> "); }
;

```

A última parte da gramática é o nível superior, que reconhece uma lista de declarações e funções.

declarações de instrução. Como antes, o nível superior avalia a AST em busca de uma instrução e imprime o

resultado, e então libera a AST. Uma definição de função é simplesmente salva para uso futuro.

Recuperação de erros básicos do analisador sintático

A última regra no analisador sintático oferece uma pequena margem de erro. Devido a

Devido à forma como os analisadores do Bison funcionam, raramente vale a pena o esforço de tentar corrigir erros, mas é possível.

pelo menos é possível recuperar para um estado em que o analisador possa continuar. O pseudo- especial

Um erro de token indica um ponto de recuperação de erro. Quando um analisador sintático do Bison encontra um erro,

começa a descartar símbolos da pilha do analisador sintático até chegar a um ponto em que um

O token de erro seria válido; então, ele descarta os tokens de entrada até encontrar um que possa ser alterado.

em seu estado atual e, em seguida, continua a análise a partir daí. Se a análise falhar novamente, ela

descarta mais símbolos da pilha e tokens de entrada até que possa retomar a análise ou o

A pilha está vazia e a análise falha. Para evitar uma cascata de mensagens de erro enganosas, o

O analisador sintático normalmente suprime quaisquer mensagens de erro de análise após a primeira, até que tenha...

Deslocou com sucesso três tokens em sequência. A macro yyerrok em uma ação informa ao

O analisador sintático indica que a recuperação foi concluída, portanto, mensagens de erro subsequentes serão geradas.

Embora seja possível, em princípio, adicionar muitas regras de erro para tentar evitar muitos erros.

Na prática, em relação à recuperação, é raro haver mais de uma ou duas regras de erro. O token de erro.

Uma calculadora avançada | 67

Faça o download em Boykma.Com

é quase sempre usado para ressincronizar em um caractere de pontuação em uma recursão de nível superior.

regra sive como fazemos aqui.

Se os símbolos descartados na recuperação de erros da pilha tiverem valores que apontam para

Com o armazenamento alocado, o processo de recuperação de erros causará vazamento de espaço, já que os valores descartados serão utilizados.

nunca são libertados. Neste exemplo, não nos preocupamos com isso, mas o bison fornece um

recurso para instruir o analisador a chamar seu código para liberar valores descartados, descrito em

Capítulo 6.

O Analisador Lexar Avançado de Calculadoras

Exemplo 3-7. Calculadora avançada léxica fb3-2.l

```
/* reconhecer tokens para a calculadora */
```

```
%option noyywrap nodefault yylineno
```

```
%{
```

```
# incluir "fb3-2.h"
```

```
#include "fb3-2.tab.h"
```

```
%}
```

```
/* expoente de ponto flutuante */
```

```
EXP ([Ee][-+]?[0-9]+)
```

```
%%
```

```
/* operações de caractere único */
```

```
"+" |
```

```
"-" |
```

"*" |

"/" |

"=" |

"|" |

"," |

"." |

"(" |

")" { return yytext[0]; }

/* operações de comparação, todas são um token CMP */

">" { yylval.fn = 1; return CMP; }

"<" { yylval.fn = 2; return CMP; }

"<>" { yylval.fn = 3; return CMP; }

"==" { yylval.fn = 4; return CMP; }

">=" { yylval.fn = 5; return CMP; }

"<=" { yylval.fn = 6; return CMP; }

/* palavras-chave */

"se" { retorne SE; }

"então" { retorne ENTÃO; }

"else" { return ELSE; }

"enquanto" { retorne ENQUANTO; }

"fazer" { retornar FAZER; }

"let" { return LET; }

Faça o download em Boykma.Com

```
/* funções internas */

"sqrt" { yylval.fn = B_sqrt; return FUNC; }

"exp" { yylval.fn = B_exp; return FUNC; }

"log" { yylval.fn = B_log; return FUNC; }

"print" { yylval.fn = B_print; return FUNC; }

/* nomes */

[a-zA-Z][a-zA-Z0-9]* { yylval.s = lookup(yytext); return NAME; }

[0-9]+ "." [0-9]* { EXP }? |

"."? [0-9]+ { EXP }? { yylval.d = atof(yytext); return NUMBER; }

"/" "." *

[ \t] /* ignorar espaços em branco */

\n { printf("c> "); } /* ignorar continuação de linha */

\n { return EOL; }

. { yyerror("Caractere misterioso %c\n", *yytext); }

%%
```

O analisador léxico, mostrado no Exemplo 3-7, adiciona algumas novas regras ao exemplo anterior.

Existem alguns novos operadores de caractere único. Os seis operadores de comparação retornam um

Token CMP com um valor lexical para distingui-los.

As seis palavras-chave e as quatro funções integradas são reconhecidas por padrões literais. Nota:

que eles precisam preceder o padrão geral para corresponder a um nome, de forma que sejam correspondidos.

em vez do padrão geral. O padrão de nome procura o nome no símbolo.

tabela e retorna um ponteiro para o símbolo.

Como antes, uma nova linha (EOL) marca o fim de uma string de entrada. Uma vez que uma função ou expressão

Se a frase for muito longa para ser digitada em uma única linha, permitimos linhas de continuação. Um novo analisador léxico.

A regra corresponde a uma barra invertida e uma nova linha e não retorna nada ao analisador sintático, tornando-se

A continuação é invisível para o analisador sintático. Mas exibe uma mensagem para o usuário.

Palavras Reservadas

Nessa gramática, as palavras `if`, `then`, `else`, `while`, `do`, `let`, `sqrt`, `exp`, `log` e `print` são

Reservado e não pode ser usado como símbolo de usuário. Se você deseja permitir que os usuários usem

O uso do mesmo nome para duas coisas no mesmo programa é discutível. Por um lado,

pode tornar os programas mais difíceis de entender, mas, por outro lado, os usuários são

obrigados a inventar nomes que não entrem em conflito com os nomes reservados.

Qualquer uma das duas pode ser levada ao extremo. COBOL tem mais de 300 palavras reservadas, então ninguém

conseguem se lembrar de todas elas, e os programadores recorrem a convenções estranhas como começar

Adicione um dígito a cada nome de variável para garantir que não haja conflito com uma palavra reservada.

Por outro lado, o PL/I não possui palavras reservadas, então você pode escrever o seguinte:

```
SE SE = ENTÃO ENTÃO SENÃO = ENTÃO; SENÃO SENÃO = SE;
```

Uma calculadora avançada | 69

Faça o download em Boykma.Com

Os analisadores sintáticos do Bison são muito mais fáceis de escrever se você reservar as palavras-chave; caso contrário, você precisa

para projetar cuidadosamente sua linguagem de forma que, em cada ponto em que o analisador léxico estiver lendo um token,

Um nome ou uma palavra-chave são válidos, mas não ambos, e você precisa fornecer detalhes extensos.

feedback para o analisador léxico para que ele saiba o que fazer.

Tendo escrito analisadores léxicos para Fortran, que

Não possui palavras reservadas e (em suas versões clássicas) ignora todos os espaços em branco. Eu fortemente...

Recomendamos que você reserve suas palavras-chave.

Construindo e interpretando ASTs

Finalmente, temos o arquivo de código auxiliar, mostrado no Exemplo 3-8. Parte deste arquivo é o

Assim como no exemplo anterior, os campos `main` e `yyerror` permanecem inalterados e não são repetidos.

aqui.

O código principal constrói e avalia as ASTs (Árvores de Sintaxe Abstrata). Primeiro, temos o gerenciamento da tabela de símbolos...
mento, que deve ser familiar a partir dos exemplos do Capítulo 2.

Exemplo 3-8. Funções auxiliares avançadas da calculadora fb3-2func.c

```
/*
```

```
* funções auxiliares para fb3-2
```

```
*/
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <stdarg.h>
```

```
#include <string.h>
```

```
#include <math.h>
```

```
# incluir "fb3-2.h"
```

```
/* tabela de símbolos */
```

```
/* hash de um símbolo */
```

```
estático sem sinal
```

```
symhash(char *sym)
```

```
{
```

```
inteiro sem sinal hash = 0;
```

```
c sem sinal;
```

```
enquanto(c = *sym++) hash = hash*9 ^ c;
```

```
retornar hash;
```

```
}
```

símbolo de estrutura *

```
pesquisa(char* sym)
```

```
{
```

```
struct symbol *sp = &symtab[symhash(sym)%NHASH];
```

```
int scout = NHASH; /* quantos analisamos */
```

```
enquanto(--contagem >= 0) {
```

```
if(sp->name && !strcmp(sp->name, sym)) { return sp; }
```

```
if(!sp->name) { /* nova entrada */
```

```
sp->name = strdup(sym);
```

70 | Capítulo 3: Utilizando Bison

Faça o download em Boykma.Com

```
sp->valor = 0;
```

```
sp->func = NULL;
```

```
sp->syms = NULL;
```

```
retornar sp;
```

```
}
```

```
if(++sp >= symtab+NHASH) sp = symtab; /* tente a próxima entrada */
```

```
}
```

```
yyerror("estouro da tabela de símbolos\n");
```

```
abortar(); /* tentei todas, a tabela está cheia */
```

```
}
```

Em seguida, vêm os procedimentos para construir os nós da AST e as listas simbólicas. Todos eles alocam um nó.

e então preencha os campos adequadamente para o tipo de nó. Uma versão estendida de

O comando `treefree` percorre recursivamente uma AST (Árvore Sintática Abstrata) e libera todos os nós da árvore.

```
struct ast *
```

```
newast(int nodetype, struct ast *l, struct ast *r)
```

```
{
```

```
struct ast *a = malloc(sizeof(struct ast));
```

```
if(!a) {
```

```
yyerror("sem espaço");
```

```
sair(0);
```

```
}
```

```
a->nodetype = nodetype;
```

```
a->l = l;
```

```
a->r = r;
```

```
retornar a;
```

```
}
```

```
struct ast *
```

```
novonum(duplo)
```

```

{
struct numval *a = malloc(sizeof(struct numval));

se(!a) {
yyerror("sem espaço");
sair(0);
}

a->nodetype = 'K';
a->número = d;
retornar (struct ast *)a;
}

```

```

struct ast *
novocmp(int cmptype, struct ast *l, struct ast *r)
{
struct ast *a = malloc(sizeof(struct ast));

se(!a) {
yyerror("sem espaço");
sair(0);
}

```

Uma calculadora avançada | 71

Faça o download em Boykma.Com

```

a->nodetype = '0' + cmptype;
a->l = l;

```

```
a->r = r;
```

```
retornar a;
```

```
}
```

```
struct ast *
```

```
novafunc(int functype, struct ast *l)
```

```
{
```

```
struct fncall *a = malloc(sizeof(struct fncall));
```

```
se(!a) {
```

```
yyerror("sem espaço");
```

```
sair(0);
```

```
}
```

```
a->nodetype = 'F';
```

```
a->l = l;
```

```
a->functype = functype;
```

```
retornar (struct ast *)a;
```

```
}
```

```
struct ast *
```

```
nova chamada(struct symbol *s, struct ast *l)
```

```
{
```

```
struct ufncall *a = malloc(sizeof(struct ufncall));
```

```
se(!a) {
```

```
yyerror("sem espaço");
```

```

sair(0);

}

a->nodetype = 'C';

a->l = l;

a->s = s;

retornar (struct ast *)a;

}

struct ast *
novoref(struct symbol *s)
{
struct symref *a = malloc(sizeof(struct symref));
se(!a) {
yyerror("sem espaço");
sair(0);
}
a->nodetype = 'N';
a->s = s;
retornar (struct ast *)a;
}

struct ast *
newasgn(struct symbol *s, struct ast *v)
{

```

```
struct symasgn *a = malloc(sizeof(struct symasgn));
```

72 | Capítulo 3: Utilizando Bison

Faça o download em Boykma.Com

```
se(!a) {
```

```
yyerror("sem espaço");
```

```
sair(0);
```

```
}
```

```
a->nodetype = '=';
```

```
a->s = s;
```

```
a->v = v;
```

```
retornar (struct ast *)a;
```

```
}
```

```
struct ast *
```

```
novofluxo(int nodetype, struct ast *cond, struct ast *tl, struct ast *el)
```

```
{
```

```
struct flow *a = malloc(sizeof(struct flow));
```

```
se(!a) {
```

```
yyerror("sem espaço");
```

```
sair(0);
```

```
}
```

```
a->nodetype = nodetype;
```

```
a->cond = cond;
```

```
a->tl = tl;
```

```
a->el = el;
```

```
retornar (struct ast *)a;
```

```
}
```

```
/* liberar uma árvore de ASTs */
```

```
vazio
```

```
treefree(struct ast *a)
```

```
{
```

```
switch(a->nodetype) {
```

```
/* duas subárvores */
```

```
caso '+':
```

```
caso '-':
```

```
caso '*':
```

```
caso '/':
```

```
caso '1': caso '2': caso '3': caso '4': caso '5': caso '6':
```

```
caso 'L':
```

```
treefree(a->r);
```

```
/* uma subárvore */
```

```
caso '|':
```

```
caso 'M': caso 'C': caso 'F':
```

```
treefree(a->l);
```

```
/* nenhuma subárvore */
```

caso 'K': caso 'N':

quebrar;

caso '=':

livre(((struct symasgn *)a)->v);

quebrar;

/* até três subárvores */

Uma calculadora avançada | 73

Faça o download em Boykma.Com

caso 'l': caso 'W':

livre(((struct flow *)a)->cond);

if(((struct flow *)a)->tl) treefree(((struct flow *)a)->tl);

se(((struct flow *)a)->el) treefree(((struct flow *)a)->el);

quebrar;

padrão: printf("erro interno: liberar nó ruim %c\n", a->nodetype);

}

free(a); /* sempre libera o próprio nó */

}

struct symlist *

newsymlist(struct symbol *sym, struct symlist *next)

{

struct symlist *sl = malloc(sizeof(struct symlist));

se(!sl) {

```

yyerror("sem espaço");

sair(0);

}

sl->sym = sym;

sl->next = próximo;

retornar sl;

}

/* libera uma lista de símbolos */

vazio

symlistfree(struct symlist *sl)
{
    struct symlist *nsl;
    enquanto(sl) {
        nsl = sl->próximo;
        livre(sl);
        sl = nsl;
    }
}

```

O núcleo da calculadora é o `eval`, que avalia uma AST construída no analisador sintático.

Seguindo a prática em C, as comparações retornam 1 ou 0 dependendo se a comparação...

A parison é bem-sucedida e os testes em if/then/else e while/do tratam qualquer valor diferente de zero como verdadeiro.

Para expressões, utilizamos a conhecida busca em profundidade na árvore para calcular o valor. Uma AST (Árvore Sintática Abstrata)

facilita a implementação de if/then/else: Avalie a AST da condição para determinar-

Decida qual caminho seguir e, em seguida, avalie a AST para o caminho a ser percorrido.

Avaliar loops while/do: um loop em eval avalia a AST da condição e, em seguida, a AST do corpo.

Repetindo enquanto a condição da AST permanecer verdadeira. Qualquer AST que faça referência a variáveis.

Os valores que são alterados por uma atribuição terão um novo valor cada vez que forem avaliados.

```
static double callbuiltin(struct fncall *);
```

```
static double calluser(struct ufncall *);
```

```
dobro
```

74 | Capítulo 3: Utilizando Bisonte

Faça o download em Boykma.Com

```
eval(struct ast *a)
```

```
{
```

```
    duplo v;
```

```
    se(!a) {
```

```
        yyerror("erro interno, avaliação nula");
```

```
        retornar 0.0;
```

```
}  
  
switch(a->nodetype) {  
  
/* constante */  
  
caso 'K': v = ((struct numval *)a)->número; interromper;  
  
/* referência de nome */  
  
caso 'N': v = ((struct symref *)a)->s->value; break;  
  
/* atribuição */  
  
caso '=': v = ((struct symasgn *)a)->s->value =  
eval(((struct symasgn *)a)->v); break;  
  
/* expressões */  
  
caso '+': v = eval(a->l) + eval(a->r); break;  
  
caso '-': v = eval(a->l) - eval(a->r); break;  
  
caso '*': v = eval(a->l) * eval(a->r); break;  
  
caso '/': v = eval(a->l) / eval(a->r); break;  
  
caso '|': v = fabs(eval(a->l)); break;  
  
caso 'M': v = -eval(a->l); interromper;  
  
/* comparações */  
  
caso '1': v = (eval(a->l) > eval(a->r))? 1 : 0; interromper;  
  
caso '2': v = (eval(a->l) < eval(a->r))? 1 : 0; interromper;  
  
caso '3': v = (eval(a->l) != eval(a->r))? 1 : 0; interromper;  
  
caso '4': v = (eval(a->l) == eval(a->r))? 1 : 0; interromper;  
  
caso '5': v = (eval(a->l) >= eval(a->r))? 1 : 0; interromper;
```

caso '6': v = (eval(a->l) <= eval(a->r))? 1 : 0; interromper;

/* fluxo de controle */

/* Expressões nulas são permitidas na gramática, portanto, verifique-as */

/* se/então/senão */

caso 'l':

if(eval(((struct flow *)a)->cond) != 0) { verifique a condição

se ((struct flow *)a)->tl) { o ramo verdadeiro

v = eval(((struct flow *)a)->tl);

} outro

v = 0.0; /* um valor padrão */

} outro {

se ((struct flow *)a)->el) { o ramo falso

v = eval(((struct flow *)a)->el);

} outro

v = 0.0; /* um valor padrão */

}

quebrar;

/* enquanto/faça */

Uma calculadora avançada | 75

Faça o download em Boykma.Com

caso 'W':

v = 0.0; /* um valor padrão */

```

se( ((struct flow *)a)->tl) {
while( eval(((struct flow *)a)->cond) != 0) avalie a condição
v = eval(((struct flow *)a)->tl); avalia as instruções de destino
}
break; /* o valor da última instrução é o valor do while/do */
/* lista de declarações */
caso 'L': eval(a->l); v = eval(a->r); break;
caso 'F': v = callbuiltin((struct fncall *)a); break;
caso 'C': v = calluser((struct ufncall *)a); break;
padrão: printf("erro interno: nó inválido %c\n", a->nodetype);
}
retornar v;
}

```

Avaliando funções na calculadora

As partes mais complexas do código no avaliador lidam com funções. Funções internas são relacionadas a...

Extremamente simples: eles determinam qual função é e chamam um código específico para

executar a função.

estático duplo

chamarbuiltin(struct fncall *f)

{

enum bifs functype = f->functype;

```
double v = eval(f->l);  
switch(func_type) {  
    caso B_sqrt:  
        retornar sqrt(v);  
    caso B_exp:  
        retornar exp(v);  
    caso B_log:  
        retornar log(v);  
    caso B_print:  
        printf("= %4.4g\n", v);  
        retornar v;  
    padrão:  
        yyerror("Função interna desconhecida %d", func_type);  
        retornar 0.0;  
}  
}
```

Funções definidas pelo usuário

A definição de uma função consiste no nome da função, uma lista de argumentos fictícios,

e uma AST que representa o corpo da função. Definir a função simplesmente salva

a lista de argumentos e a AST na entrada da tabela de símbolos da função, substituindo qualquer anterior.

versão.

```
/* define uma função */
```

vazio

```
dodef(struct symbol *name, struct symlist *syms, struct ast *func)
```

```
{
```

```
if(name->syms) symlistfree(name->syms);
```

```
if(name->func) treefree(name->func);
```

```
nome->syms = syms;
```

```
nome->func = func;
```

```
}
```

Suponha que você defina uma função para calcular o máximo de seus dois argumentos:

> seja $\text{max}(x,y) = \text{se } x \geq y \text{ então } x; \text{ senão } y;;$

> $\text{máx}(4+5,6+7)$

A função possui dois argumentos fictícios, x e y. Quando a função é chamada,

O avaliador faz isso:

1. Avalie os argumentos reais, 4+5 e 6+7 neste caso.

2. Salve os valores atuais dos argumentos fictícios e atribua os valores dos argumentos reais.

argumentos para eles.

3. Avalie o corpo da função, que agora utilizará os valores reais dos argumentos.

quando se refere aos argumentos fictícios.

4. Restaure os valores antigos das variáveis fictícias.

5. Retorne o valor da expressão do corpo.

O código para fazer isso conta os argumentos e aloca dois arrays temporários para o antigo.

e novos valores para os argumentos fictícios, e então executa os passos descritos anteriormente.

estático duplo

```
calluser(struct ufncall *f)
```

```
{
```

```
struct symbol *fn = f->s; /* nome da função */
```

```
struct symlist *sl; /* argumentos fictícios */
```

```
struct ast *args = f->l; /* argumentos reais */
```

```
double *oldval, *newval; /* valores de argumentos salvos */
```

```
duplo v;
```

```
int nargs;
```

```
int i;
```

```
se(!fn->func) {
```

```
yyerror("chamada para função indefinida", fn->name);
```

```
retornar 0;
```

```
}
```

```
/* contar os argumentos */
```

```
sl = fn->syms;
```

```
for(nargs = 0; sl; sl = sl->next)
```

```
nargs++;
```

Uma calculadora avançada | 77

Faça o download em Boykma.Com

```
/* prepare-se para salvá-los */
```

```
oldval = (double *)malloc(nargs * sizeof(double));
```

```
novoval = (double *)malloc(nargs * sizeof(double));
```

```
if(!oldval || !newval) {
```

```
yyerror("Sem espaço em %s", fn->name); return 0.0;
```

```
}
```

```
/* avaliar os argumentos */
```

```
para(i = 0; i < nargs; i++) {
```

```
se(!args) {
```

```
yyerror("poucos argumentos na chamada para %s", fn->name);
```

```
liberar(valor antigo); liberar(valor novo);
```

```
retornar 0.0;
```

```
}
```

```
if(args->nodetype == 'L') { /* se este for um nó de lista */
```

```
novoval[i] = eval(args->l);
```

```
args = args->r;
```

```

} else { /* se for o fim da lista */

    novoval[i] = eval(args);

    args = NULL;

}

}

/* Salvar valores antigos de variáveis fictícias, atribuir novos valores */

sl = fn->syms;

para(i = 0; i < nargs; i++) {

    struct symbol *s = sl->sym;

    oldval[i] = s->valor;

    s->valor = novoval[i];

    sl = sl->próximo;

}

livre(novovalor);

/* avaliar a função */

v = eval(fn->func);

/* Devolva os valores reais das variáveis fictícias */

sl = fn->syms;

para(i = 0; i < nargs; i++) {

    struct symbol *s = sl->sym;

    s->valor = oldval[i];

    sl = sl->próximo;

```

```
}  
livre(valor antigo);  
retornar v;  
}
```

78 | Capítulo 3: Utilizando Bisonte

Faça o download em Boykma.Com

Usando a calculadora avançada

Esta calculadora é suficientemente flexível para realizar alguns cálculos úteis. O exemplo 3-9 mostra...

a função `sq` para calcular raízes quadradas iterativamente usando o método de Newton, bem como

Uma função auxiliar chamada `avg` para calcular a média de dois números.

Exemplo 3-9. Calculando raízes quadradas com a calculadora

> seja `sq(n)=e=1; enquanto |((t=n/e)-e)>.001 faça e=avg(e,t);;`

quadrado definido

> seja `avg(a,b)=(a+b)/2;`

Média definida

> `sq(10)`

= 3,162

> `sqrt(10)`

= 3,162

> `sq(10)-sqrt(10)`

= 0,000178 preciso, com uma margem de erro melhor que o limite de 0,001

Exercícios

1. Experimente algumas variantes da sintaxe na calculadora aprimorada. No exame anterior-

Por exemplo, a função `sq` precisa terminar com dois pontos e vírgulas para fechar o loop `while`.

e a declaração ``let``, que é bem desajeitada. Você pode alterar a sintaxe para torná-la mais fácil?

Fica mais intuitivo? Se você adicionar símbolos de fechamento às declarações condicionais `if/then/else/`

`fi` e loops `while/do/done`, você pode tornar a sintaxe das listas de instruções mais

flexível?

2. No último exemplo, as funções do usuário avaliam todos os argumentos reais e os colocam no lugar.

em um array temporário e, em seguida, atribuí-los aos argumentos fictícios. Por que não?

Basta fazer uma de cada vez e definir as variáveis fictícias à medida que os valores reais forem avaliados?

Exercícios | 79

Faça o download em Boykma.Com